🎀 Chuleta de C 🎀

Todo lo básico, en orden y con ejemplos · incluye punteros, `malloc`, chivato y burbuja

1 · Esqueleto de un programa

```c
#include <stdio.h>   // printf, scanf

int main(void) {
    printf("Hola!\n");
    return 0;          // 0 = todo bien
}
```

- **Toda instrucción acaba en `;`** (las llaves no).
- Los bloques van entre `{ }`. La sangría no es obligatoria como en Python, pero úsala siempre.
- Compilar y ejecutar (terminal): `gcc prog.c -o prog` → `./prog`
- Comentarios: `// una línea` o `/* varias */`

2 · Variables, tipos y entrada/salida

🫙 **Analogía:** una variable es un **tarro con etiqueta**. En C tienes que decir **qué cabe** en el tarro antes de usarlo (el tipo), y no puedes meter otra cosa.

Tipo	Guarda	Ejemplo	`printf`	`scanf`
`int`	entero	`int edad = 19;`	`%d`	`%d`
`float`	decimal (menos preciso)	`float x = 2.5;`	`%f`	`%f`
`double`	decimal (más preciso)	`double pi = 3.1416;`	`%f` / `%.2f`	`%lf` ⚠️
`char`	un carácter	`char letra = 'a';`	`%c`	`" %c"` ⚠️
`char[]`	texto (cadena)	`char nom[20];`	`%s`	`%s` (sin `&`)

```c
int n;
printf("Dame un numero: ");
scanf("%d", &n);                 // & = "guárdalo en la dirección de n"
printf("El doble es %d\n", 2 * n);
printf("Media: %.2f\n", media); // %.2f = 2 decimales
```

⚠️ **Ojo:** en `scanf` casi todo lleva `&` delante (menos las cadenas). Para un `double` se lee con `%lf`. Para un `char`, pon un espacio: `scanf(" %c", &c);` (si no, se "come" el Enter anterior).

3 · Operadores

Aritméticos

- `+ - * /` y `%` (resto, solo enteros): `7 % 3` → `1`
- **División entera**: `7 / 2` → `3` (¡se pierde el ,5!) `7 / 2.0` → `3.5` · `(double) a / b` → decimal
- Atajos: `i++` (suma 1), `i--`, `x += 5`, `x *= 2`

Comparación y lógicos

- `==` igual · `!=` distinto · `< > <= >=`
- `&&` "y" · `||` "o" · `!` "no"
- En C, **0 es falso** y **cualquier otro número es verdadero**.

⚠️ **El clásico: `=` guarda, `==` compara.** `if (x = 5)` no compara: mete un 5 en x y siempre es verdadero.

4 · Condicionales

```c
if (nota >= 9) {
    printf("Sobresaliente\n");
} else if (nota >= 5) {
    printf("Aprobado\n");
} else {
    printf("Suspenso\n");
}
```

```c
switch (opcion) {     // solo int o char
    case 1:
        printf("Uno\n");
        break;        // ¡sin break sigue!
    case 2:
        printf("Dos\n");
        break;
    default:          // "ninguno de los anteriores"
        printf("Otro\n");
}
```

5 · Bucles: ¿cuál uso?

🏃 **Analogía: for** = "da 10 vueltas a la pista" (sabes cuántas). **while** = "corre mientras no llueva" (puede que ni empieces). **do-while** = "da una vuelta y luego decide si sigues" (mínimo una).

```c
for (int i = 0; i < 5; i++) {
    printf("%d ", i);     // 0 1 2 3 4
}
```

```c
while (n != 0) {          // comprueba ANTES
    scanf("%d", &n);
}
```

```c
do {                      // comprueba DESPUÉS
    printf("Numero (1-10): ");
    scanf("%d", &n);
} while (n < 1 || n > 10); // ¡lleva ;!
```

💡 El `do-while` es perfecto para **validar datos**: pides, y si está mal, vuelves a pedir.

`break` sale del bucle · `continue` salta a la siguiente vuelta.

6 · Vectores (arrays)

🗄️ **Analogía:** un vector es una **fila de taquillas numeradas**, todas del mismo tipo. La primera es la **0**, así que con 5 taquillas la última es la **4**.

```c
int v[5] = {4, 8, 15, 16, 23};
v[0] = 7;             // primera
v[4] = 1;             // última (N-1)

// Leer N números
for (int i = 0; i < n; i++) {
    scanf("%d", &v[i]);
}
```

```c
// Mostrarlos
for (int i = 0; i < n; i++) {
    printf("%d ", v[i]);
}
printf("\n");
```

⚠️ `v[5]` en un vector de 5 **se sale**. C no avisa: lee basura o el programa peta. Con tamaño fijo (`int v[100]`), comprueba que `N` no pase de 100.

7 · Cadenas de texto

Una cadena es un vector de `char` que termina en `'\0'` (el "punto final" invisible). Por eso `char nom[20]` guarda como mucho **19** letras.

```c
#include <string.h>
char nom[20];
scanf("%s", nom);     // sin &, para en el espacio
```

Función	Qué hace
`strlen(s)`	longitud (sin el `'\0'`)
`strcpy(a, b)`	copia b en a
`strcat(a, b)`	pega b al final de a
`strcmp(a, b)`	**0 si son iguales**

⚠️ Las cadenas no se comparan con `==` ni se copian con `=` .

8 · Funciones

```c
int suma(int a, int b);   // prototipo (antes del main)

int main(void) {
    int r = suma(3, 4);   // r = 7
    printf("%d\n", r);
    return 0;
}

int suma(int a, int b) {  // definición
    return a + b;
}
```

- **Tipo de retorno** delante (`int` , `double` …). Si no devuelve nada: `void` .
- `return` devuelve el valor y **sale** de la función.
- Las variables de dentro solo existen dentro.

📄 **Paso por valor:** la función recibe una **fotocopia** de los datos. Si la pinta, el original no cambia. Para cambiar el original… ¡punteros! 👇

9 · Punteros

🏠 **Analogía:** cada variable vive en una **casa** de la memoria, y cada casa tiene una **dirección**. Un puntero es un papelito donde apuntas **la dirección**, no lo que hay dentro.
`&x` → "¿dónde vive x?" (su dirección) · `*p` → "ve a la casa apuntada y mira/cambia lo que hay"

```c
int x = 5;
int *p = &x;     // p guarda la dirección de x

printf("%d\n", *p);  // 5 (lo que hay en la casa)
*p = 10;             // cambio lo de la casa...
printf("%d\n", x);   // ...¡y x vale 10!
```

El `*` significa dos cosas: en la **declaración** (`int *p`) dice "p es un puntero"; **al usarlo** (`*p`) dice "ve a esa dirección".

Paso por referencia

```c
void doblar(int *n) {
    *n = *n * 2;     // cambia el ORIGINAL
}

int main(void) {
    int x = 4;
    doblar(&x);      // le das la dirección
    printf("%d\n", x); // 8
    return 0;
}
```

¡Por eso `scanf` lleva `&` ! Necesita la dirección para poder escribir en tu variable.

💜 **Punteros y vectores:** el nombre de un vector es **la dirección de su primera casilla**. Por eso `v[i]` es lo mismo que `*(v + i)` , y por eso una función que recibe un vector (`void f(int v[], int n)`) **sí modifica el original**: le llega la dirección, no una fotocopia.

10 · Memoria dinámica: malloc y free

🍽️ **Analogía:** `int v[100]` es reservar **siempre** mesa para 100. Con `malloc` llamas al restaurante **cuando ya sabes cuántos sois** y pides mesa para justo N. Al irte, **devuelves la mesa** con `free` .

```c
#include <stdlib.h>      // malloc, free

int n;
scanf("%d", &n);
int *v = malloc(n * sizeof(int));
if (v == NULL) {           // ¿restaurante lleno?
    printf("Sin memoria\n");
    return 1;
}
for (int i = 0; i < n; i++) {
    scanf("%d", &v[i]);    // se usa como un vector
}
free(v);                   // devolver la mesa
```

- `malloc` pide memoria **en bytes**: por eso `n * sizeof(int)` , no `malloc(n)` .
- Devuelve la **dirección** del bloque → se guarda en un **puntero**.
- Si no hay memoria, devuelve `NULL` : compruébalo siempre.
- Cada `malloc` necesita su `free` .

⚠️ **Errores típicos:** olvidar el `free` (**fuga de memoria**), usar el vector **después** del `free` , o hacer `free` dos veces.

11 · El método del chivato (bandera / flag)

👀 **Analogía:** la profe sale y le dice a un alumno: *"Vigila, y si alguien habla, me lo dices"*. El chivato empieza **callado**, **cambia** si pasa algo, y al final **le preguntas**.

Receta en 3 pasos:

- ① **Antes del bucle:** valor inicial (lo que supones).
- ② **Dentro:** si pasa lo que vigilas, lo cambias.
- ③ **Después:** miras cuánto vale y decides.

⚠️ **Errores típicos:**
- Inicializarlo **dentro** del bucle: se "olvida" en cada vuelta.
- Ponerlo otra vez al valor inicial en un `else`: **borra el aviso**. El chivato solo cambia en **una dirección**.

```c
int primo = 1;              // ① suponemos que SÍ
if (n < 2) primo = 0;

for (int i = 2; i < n; i++) {
    if (n % i == 0) {
        primo = 0;          // ② ¡divisor! avisa
        break;              // ya no hace falta seguir
    }
}

if (primo == 1)             // ③ le preguntamos
    printf("Es primo\n");
else
    printf("No es primo\n");
```

12 · Ordenar: burbuja y burbuja con chivato

🫧 **Analogía:** una fila de alumnos que se ordena por altura. Recorres la fila mirando **parejas de vecinos**: si el de la izquierda es más alto, **se cambian**. En cada pasada el más alto "sube" al final, como una burbuja.

🔁 Burbuja "a secas"

- Hace **siempre N-1 pasadas**, con dos `for` anidados.
- En cada pasada compara `v[j]` con `v[j+1]` y, si están al revés, los intercambia.
- Problema: aunque el vector ya esté ordenado, **sigue dando vueltas** para nada.

👀 Burbuja con chivato

- Compara vecinos exactamente igual.
- El chivato vigila: **¿ha habido algún intercambio en esta pasada?** Al empezar cada pasada se pone a "no".
- Si una pasada entera acaba **sin cambios**, ya está ordenado → **para**. Como hay que hacer al menos una pasada, encaja un **do-while**.

🥤 **El intercambio:** para cambiar el contenido de dos vasos necesitas un **tercer vaso vacío** (una variable `aux`): primero vacías uno en el auxiliar, luego el otro en el primero, y el auxiliar en el segundo.

Ejemplo dibujado: ordenar 3 1 2 5 4

Pasada	Comparaciones (parejas de vecinos)	Resultado	¿Cambios?
1	(3,1)↔ · (3,2)↔ · (3,5) ok · (5,4)↔	1 2 3 4 5	Sí → otra pasada
2	(1,2) ok · (2,3) ok · (3,4) ok · (4,5) ok	1 2 3 4 5	No → ¡FIN!

Con chivato: **2 pasadas**. Con la burbuja normal habría hecho **4** (N-1), dos de ellas inútiles. Esa es toda la gracia del chivato 😉

13 · Lista de errores típicos ✅

- ☐ ¿Todas las instrucciones acaban en `;`? (¡y el `do-while`!)
- ☐ ¿`==` para comparar y `=` para guardar?
- ☐ ¿`&` en los `scanf` (menos cadenas)?
- ☐ ¿`%lf` para leer `double`?
- ☐ ¿División entera donde querías decimales?
- ☐ ¿Los índices van de **0** a **N-1**? (`i < n`, no `i <= n`)
- ☐ ¿Comparar `v[j+1]` sin salirte del vector?
- ☐ ¿Variables inicializadas antes de usarlas (contadores, sumas, chivatos)?
- ☐ ¿`break` en cada `case`?
- ☐ ¿Un `free` por cada `malloc`?

✨ ¡Tú puedes! Error → corrección → error más pequeño → ¡funciona! ✨